

Verilog (week 2): Brand New Day

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab



Back to Basics: Module Definition

- Module definition syntax

```
module priority_decoder_4to2 (  
    input wire [3:0] in,          // 4 input request lines  
    output reg [1:0] code,        // encoded index of highest-priority input  
    output reg          valid     // 1 if any input is asserted  
> ); ...  
  
endmodule
```

- `input/output` are keywords to define direction of port (`inout` port can be used for both)
- `// ...` and `/* */` are for comments
- `wire/reg` define data types and `[N:0]` defines size; we will get to them later
- Each module definition must define ports (inputs & outputs) in parentheses
- Module definition ends with `endmodule`
- **Note:** Curly braces are replaced with explicit `begin, end...`-style statements in Verilog

Back to Basics: Instantiation

- define variables for **input/output**
for eg: `req_lines`, `priority_code`,
`req_valid`
- instantiate it with a **unique instance name**
for eg: `pdec_inst` is the unique identifier for this
instance of `priority_decoder_4to2`
- wire in the inputs/outputs using **port names**
for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg [3:0] req_lines;    // input - reg
wire [1:0] priority_code; // output - wire
wire req_valid;        // output - wire

priority_decoder_4to2 pdec_inst (
    .in      (req_lines),
    .code    (priority_code),
    .valid    (req_valid)
);
```

Back to Basics: Can you C it?

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to basics: One isn't good enough

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment:**

`data = 4'b0110` (full assignment)

`data[1:0] = 2'b10` (multi-bit assignment)

`data[3:0] = {2'b01, 2'b10}` (using concat operator)

> this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

for eg: `data[high:low]` will read bits `low, low+1, ... high-1, high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Back to Basics: More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **integer** – plain whole number variable, mostly used to count things (like loop iterations)
for eg: `integer i; for (i = 0; i < 16; i = i + 1) .`
- **parameter** – a named constant you can **reconfigure** from outside the module
for eg: `parameter WIDTH = 8` – can write variable-width modules using this
- **real** – a decimal/floating-point number; **only used in simulation/testing**
for eg: `real voltage = 3.3 .`
- **time** – stores a **simulation/testing timestamp**, usually paired with `$time`
for eg: `time start_time; start_time = $time; .`
- **localparam** – a named constant that is **fixed inside a module** – used to store helper values
for eg: `localparam OP_ADD = 4'b0000; .`

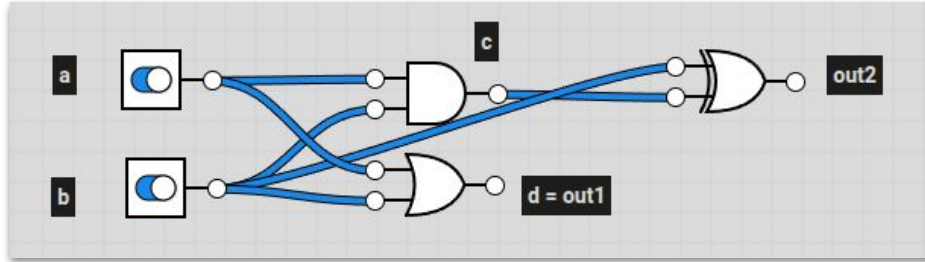
And two roads diverged...

- You will think of digital logic as either combinational or sequential
 - **Combinational** — output depends only on current inputs; no memory, no clock
 - **Sequential** — output depends on current inputs and past state; needs a clock
- Let's see them in more detail

.. and the other, just as fair ..

- Combinational circuits are a **direct mapping** from **input signals** to **output signals**.

- Eg:



- Regardless of what the previous input was the output for a given input is the same.
- You can represent them using simple gates – AND, OR, NOT – chained together
- There is **no memory** and **no notion of a clock**
- We can say the output is “always” the same depending on the input
- Eg: using assign

```
assign sum = a ^ b ^ cin;
```

using always @(*)

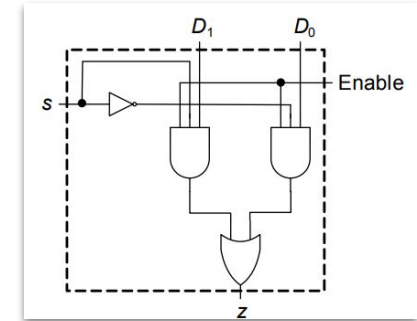
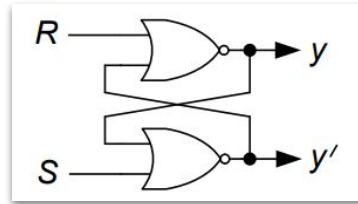
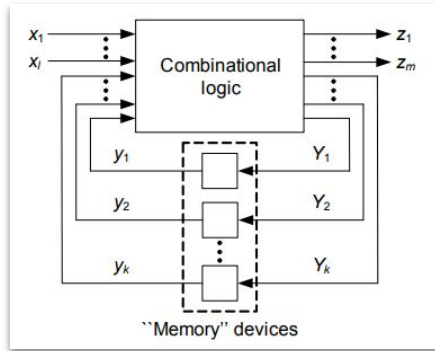
```
always @(*) begin
    if (s)
        o = a;
    else
        o = b;
end
```

Theory Terms

- Recall the following from CS230:
 - **Clock:** A periodic signal (typically square wave) that triggers updates in sequential circuits.
 - **Level-triggered:** A circuit that responds continuously **while a control signal is high** (or low).
 - **Edge-triggered:** A circuit that responds only at **an instant** — when the clock transitions from **0→1 (posedge)** or **1→0 (negedge)**
- **D Flip-Flop (edge-triggered):** A circuit that samples its input on a clock edge and holds that value until the next edge
- **T Flip-Flop (edge-triggered):** A circuit that toggles its output (flips $0 \leftrightarrow 1$) when both the clock edge arrives and the toggle input is 1

.. I took the one less travelled by

- Sequential circuits are a **mapping from (input, previous state) $\mapsto$ (output, next state)**
- Eg: general sequential circuit = latches + combinational logic



- Think of registers/latches as **abstract “data stores”** and treat the seq circuit as combinational logic applied in a **clocked** (update based clock edge/level) and **deferred** (one write per clock edge) manner
- **What do we need?** (1) registers (2) clock-based triggering (3) deferring of updates

Sequential is All You Need

- We intend to cover how verilog provides these 3 abstractions:
 - a. **Registers:** Defining `reg` variables for “memory”
 - b. **Clock-based triggering:** Using `always` with suitable trigger list eg: `always @(posedge clk)`
 - c. **Deferring of updates:** Using non-blocking assignment `<=` to defer writes until all blocks have read

Data Types Revisited

Verilog supports two data types for these purposes:

- **wire** – carries a value **continuously** from whatever drives it; has no memory of its own
 - **assign** is used for **continuous assignment** to a wire. It cannot be used for register data.
 - Eg: `assign q_bar = ~q_reg;`
- **reg** – a variable that **holds its value** until explicitly assigned inside a procedural block
 - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
 - Can only be assigned inside of `always @(...)` blocks
 - Usage: `always @(trigger-list)`
 - Eg: `always @(posedge clk)` executes on **rising edge of clk** signal -> **sequential**
 - Eg: `always @(*)` executes whenever **any input changes** -> **combinational**

Always there for you

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (=) is used
- written inside **always @(*)** blocks
- the blocking statements run in **program order**
- Eg: some random combinational logic here

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (<=) is used
- written inside **always @(posedge clk)** blocks (or a variant)
- Eg: naive 2-bit (modulo 4) counter

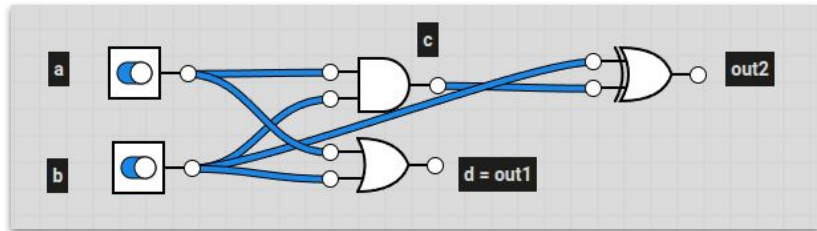
```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

Always there for you (still)

Combinational

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

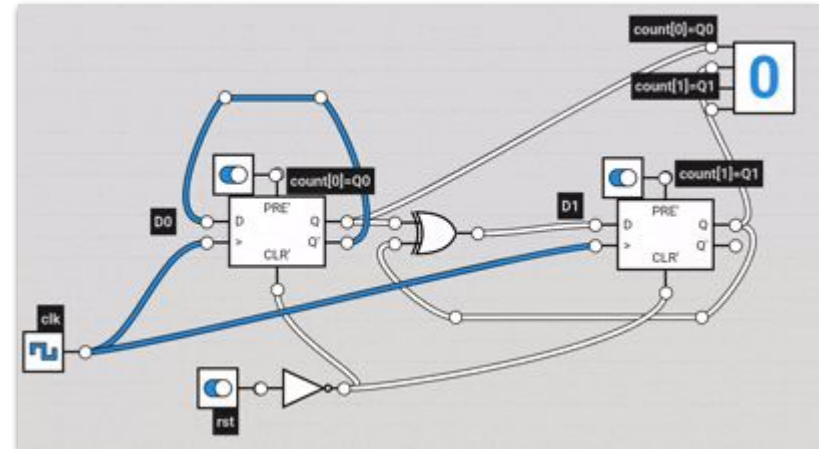
- Circuit will respect the program order for computation



Sequential

```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

- Circuit will respect the clock-based update



Quiz

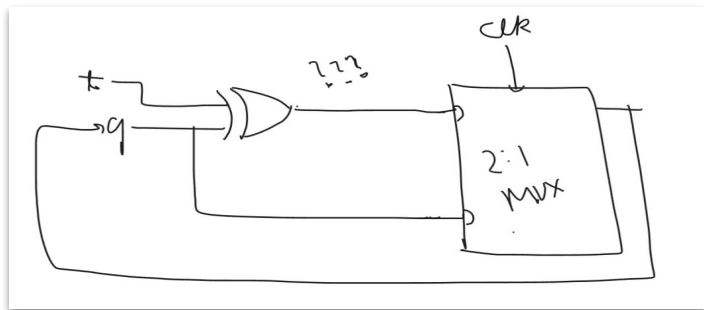
- Is there something wrong with this code snippet? (Assume usual T Flip Flop semantics)

```
module tff_wire (  
    input clk,  
    input t,  
    output wire q  
);  
    assign q = clk ? (t ^ q) : q;  
endmodule
```

Answer: **wire** isn't enough

- Recall from CS230:
 $Q_{t+1} = T \oplus Q_t$ satisfies T-flip flop semantics
- But this is **level-triggered**, not **edge-triggered**!
- Plus, looping an output back in as input is wrong.
- **Wire q cannot settle** on a value as long as **clk** is on and **t** is set (it will keep switching between 0/1):
- This leads to **undefined state (X)** for **q**

```
module tff_wire (  
    input clk,  
    input t,  
    output wire q  
);  
    assign q = clk ? (t ^ q) : q;  
endmodule
```



- **We need to use reg**

- Another issue is the lack of a reset pin but let's ignore that for now

Quiz

- Let's use a **reg** now.
- Is there something wrong with this code snippet? (Assume usual T Flip Flop semantics)

```
module tff_reg (  
    input clk,  
    input t,  
    output reg q  
);  
    always @(posedge clk) begin  
        q = t ^ q;  
    end  
endmodule
```

Answer: Is it really sequential?

- This satisfies T flip flop semantics + edge-triggered correctly
- Appears that **q** will update **exactly on posedge** (instantaneously)
- On posedge: evaluate **$t \wedge q$** , set **q** to that value, repeat.



```
module tff_reg (  
    input clk,  
    input t,  
    output reg q  
);  
    always @(posedge clk) begin  
        q = t ^ q;  
    end  
endmodule
```

- But **blocking assignment is wrong** here
- Doesn't guarantee deferring of updates, i.e., **one update per time step**

Quiz (optional)

- What's wrong with this code snippet?
 - Will `tff2.q` toggle once every 2 time periods as intended?

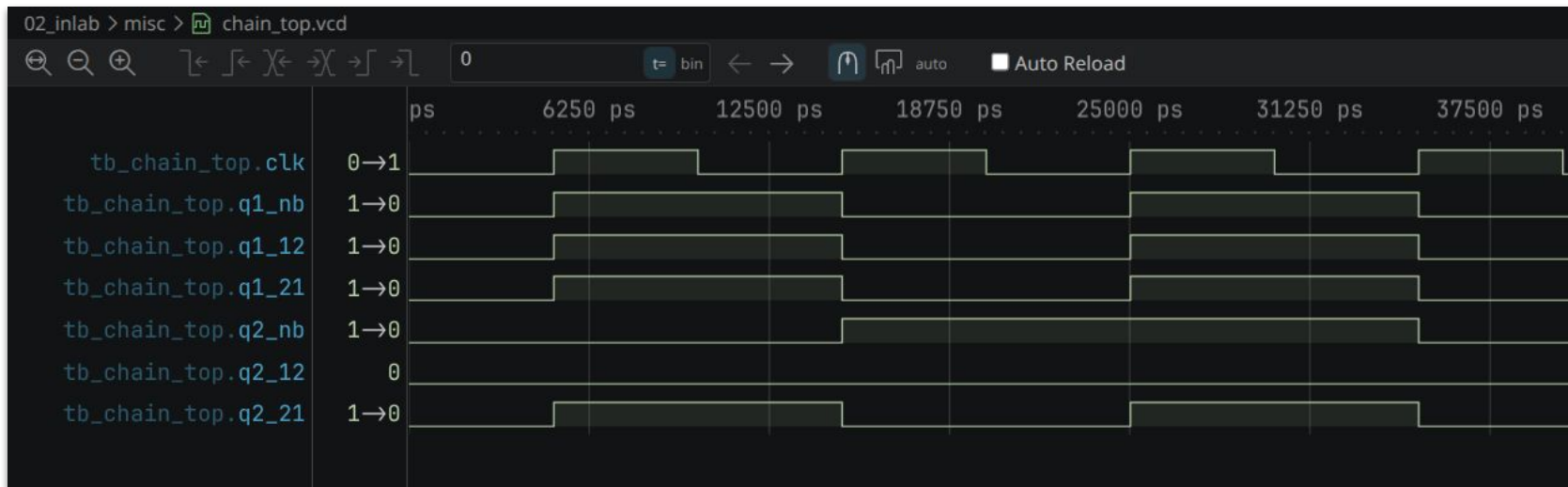
```
module chain_top (  
    input clk,  
    output q1,  
    output q2  
);  
    // always toggles      => switches once per clock time period  
    tff_reg tff1 (.clk(clk), .t(1'b1), .q(q1));  
    // toggles when q1==1  => switches once per 2 clock time periods  
    tff_reg tff2 (.clk(clk), .t(q1), .q(q2));  
endmodule
```

Answer: Don't block me

- TLDR: Use `<=` for making updates properly timed (or non-blocking) in a sequential circuit.
- Blocking assignment (`=`) updates many variables **instantly** — this violates sequential circuit semantics which requires **all writes to be deferred to the end** and **executed as one unit**
- With a single flip-flop, this doesn't show up as an issue, as we saw in `tff_reg`
- But chain two `tff_reg`'s together (`q` output of first is fed as `t` input of the second) and now **both always blocks will fire on the same rising edge**
- Since `=` writes immediately, it will **not defer writes to the end**; any `always` block can do a write first, leading to a race condition.
- Hence, output is implementation dependent. It also depends on whether `tff1` is initialized first or `tff2` is.

Answer: Don't block me

- Here `_nb` refers to the correct, non-blocking version;
And `_12` and `_21` refer to order of initialization of `tff1/2` in the wrong, blocking version
- All `q1_*` toggle per 1 time period as intended (deferring of updates is only an issue with ≥ 1 interfering always blocks)
- `q2_nb` correctly toggles every 2 time periods
- `q2_12` does not toggle at all!
- `q2_21` toggles every 1 time period!



Do it once

- `<=` stands for non-blocking assignment – this allows us to defer updates to the end and execute them together
- How it works at a high level:
 - Read all old values (occurring on right hand side first)
 - Then update all left hand side values **together**
- Why is it called “non-blocking”?

All writes are **scheduled** to happen later, i.e., it **doesn't block** any other statement trying to read the **old value**.
A blocking assignment (`=`) will write the new value immediately and block anyone else trying to read the old value.
- **Rule of thumb:** Always use `<=` inside `always @(posedge clk)` or some variant.
- **Note:** `<=` and `=` (other than `assign <wire> = statements`) can only be used in `always` or `initial` blocks.

Example: DFF

```
module dff_with_inverter (  
    input wire clk,  
    input wire rst_n,  
    input wire d,  
    output wire q,  
    output wire q_bar  
);  
  
    reg q_reg;  
  
    // ---- Sequential logic ----  
    // Output depends on current input AND past state (memory)  
    // Only updates on the clock's rising edge  
    always @(posedge clk) begin  
        if (!rst_n)  
            q_reg <= 1'b0;  
        else  
            q_reg <= d;  
        end  
  
    // ---- Combinational logic ----  
    // Output depends only on current input, no memory, no clock  
    assign q = q_reg;  
    assign q_bar = ~q_reg;  
  
endmodule
```

Example: DFF testbench

```
`timescale 1ns/1ns

module tb_dff_with_inverter;

    reg clk, rst_n, d;
    wire q, q_bar;

    dff_with_inverter dut (
        .clk(clk), .rst_n(rst_n),
        .d(d), .q(q), .q_bar(q_bar)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dff_wave.vcd");
        $dumpvars(0, tb_dff_with_inverter);

        clk = 0; rst_n = 0; d = 0;

        #12 rst_n = 1; // release reset
```

```
        initial begin
            $dumpfile("dff_wave.vcd");
            $dumpvars(0, tb_dff_with_inverter);

            clk = 0; rst_n = 0; d = 0;

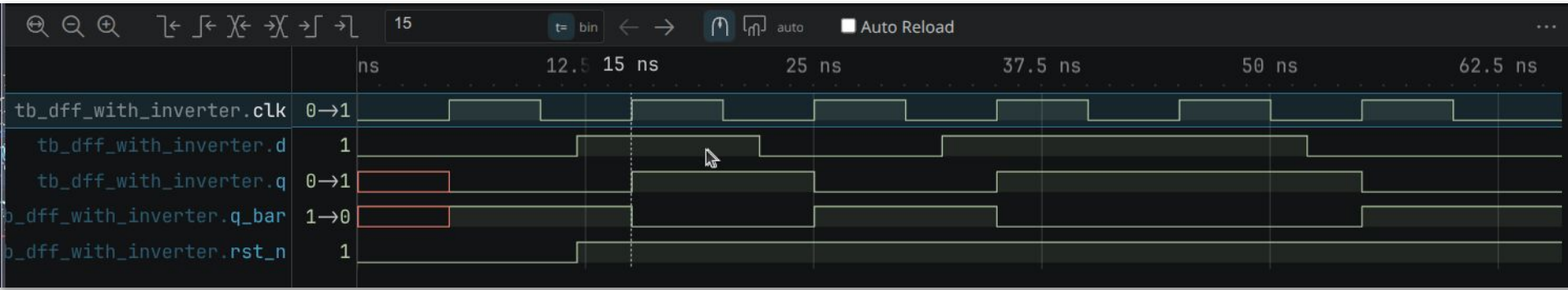
            #12 rst_n = 1; // release reset

            d = 1; #10;
            d = 0; #10;
            d = 1; #10;
            d = 1; #10;
            d = 0; #10;

            $display("Test complete.");
            $finish;
        end

        initial begin
            $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
                $time, rst_n, d, q, q_bar);
        end
    endmodule
```

Example: DFF Waveform



- An example testbench waveform for the D-flip flop implementation
- Note clock time period is 10ns
- **d** update is propagated to **q** only on **rising edge** when **rst_n** is set

It's inlab time



It's graded lab time



1. Log in to **CS231-labexam**, password will be shared
2. Extract labexam tarball and work in the extracted folder
3. Go into the same folder and run `./build-archive.sh`
4. Log in to moodle and submit into the **correct activity**: ...